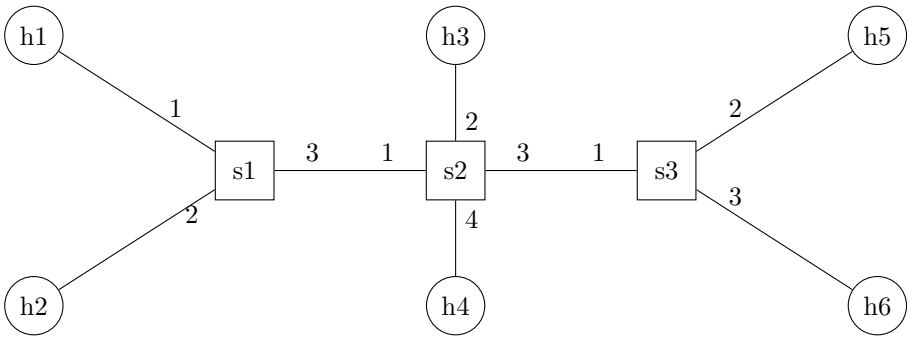
Software Defined Networking – 25 luglio 2018

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Nelle risposte usare per ogni pacchetto il seguente formato:

MAC sorgente → MAC destinazione; ethertype; IP sorgente → IP destinazione; TTL; protocollo; porta sorgente → porta destinazione;

(a) Scrivere il contenuto delle tabelle openflow dopo l'accensione di tutti i nodi.

Soluzione:

s1

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = ETH_TYPE_IP ipv4_dst = IP5 ip_proto = IPPROTO_UDP udp_dst = 123	eth_dst = MAC3 ipv4_dst = IP3 DecNwTTL Output(3)

s2

Priority	Match	Action
0	*	Output(CONTROLLER,128)
1	eth_type = ETH_TYPE_MPLS mpls_label = 1000	DecMplsTtl Output(3)

s3

Priority	Match	Action
0	*	Output(CONTROLLER,128)
1	eth_type = ETH_TYPE_MPLS mpls_label = 1000	PopMpls Output(3)

L'host h1, invia un datagramma UDP porta 7654 a h5 porta 123.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

Programma 1 (1/2) – Switch Features

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    ### table miss ###
    match = parser.OFPMatch()
    actions = [ parser.OFPActionOutput(ofproto.OFPP_CONTROLLER, 128) ]
    inst = [ parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions) ]
    mod = parser.OFPFlowMod(
        datapath=datapath, priority=0, match=match, instructions=inst)
    datapath.send_msg(mod)

   if (datapath.id == 1):
        match = parser.OFPMatch(
            eth_type = ETH_TYPE_IP,
            ipv4_dst=IP5,
            ip_proto = IPPROTO_UDP,
            udp_dst = 123)
        actions = [
            parser.OFPActionSetField(eth_dst = MAC3),
            parser.OFPActionSetField(ipv4_dst = IP3),
            parser.OFPActionDecNwTtl(), # decrementa TTL
            parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=5,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    if (datapath.id == 2):
        match = parser.OFPMatch(eth_type = ETH_TYPE_MPLS, mpls_label=1000)
        actions = [
            parser.OFPActionDecMplsTtl(),
            parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    if (datapath.id == 3):
        match = parser.OFPMatch(eth_type = ETH_TYPE_MPLS, mpls_label=1000)
        actions = [
            parser.OFPActionPopMpls(),
            parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=1,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

Programma 1 (2/2) – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (datapath.id == 2) and (pkt_tcp is not None) and (pkt_ipv4.dst==IP5):
        match = parser.OFPMatch(
            eth_type=ETH_TYPE_IP, ipv4_src=pkt_ipv4.src, ipv4_dst=IP5,
            ip_proto=IPPROTO_TCP, tcp_dst=pkt_tcp.dst_port)
        actions = [
            parser.OFPActionSetField(tcp_dst = pkt_tcp.dst_port + 1024),
            parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3, match=match,
                                instructions=inst, buffer_id=buffer_id)
        datapath.send_msg(mod)

        match = parser.OFPMatch(
            eth_type=ETH_TYPE_IP, ipv4_src=IP5, ipv4_dst=pkt_ipv4.src,,
            ip_proto=IPPROTO_TCP, tcp_src=pkt_tcp.dst_port + 1024)
        actions = [
            parser.OFPActionSetField(tcp_src = pkt_tcp.dst_port),
            parser.OFPActionOutput(in_port) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
        return

    if (datapath.id == 1) and (pkt_udp is not None) and and (pkt_ipv4.dst==IP5):
        match = parser.OFPMatch(
            eth_type=ETH_TYPE_IP, ipv4_dst=IP5,ip_proto=IPPROTO_UDP,
            udp_dst=pkt_udp.dst_port )
        actions = [
            parser.OFPActionPushMpls(), parser.OFPActionSetField(mpls_label=1000),
            parser.OFPActionOutput(3) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3, match=match,
                                instructions=inst, buffer_id=buffer_id)
        datapath.send_msg(mod)
        return

    outport = ofproto.OFPP_FLOOD
    if (datapath.id == 1) and (in_port < 3):
        outport = 3
    elif (datapath.id == 3) and (in_port > 1):
        outport = 1

    actions = [parser.OFPActionOutput(outport)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

Soluzione:
$s1 \rightarrow s2$: MAC1 → MAC3; IPv4; IP1 → IP3; 199; UDP; 7654 → 123

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow \mathcal{C}$: Packet_in(MAC1 → MAC3; IPv4; IP1 → IP3; 199; UDP; 7654 → 123)
$s2 \rightarrow s3, h3, h4$: MAC1 → MAC3; IPv4; IP1 → IP3; 199; UDP; 7654 → 123

L'host h2 apre una connessione TCP porta 1234 verso h5 porta 53.

(d) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow \mathcal{C}$: Packet_in(MAC2 → MAC5; IPv4; IP2 → IP5; 200; TCP; 1234 → 53)
$s2 \rightarrow s3, h3, h4$: MAC2 → MAC5; IPv4; IP2 → IP5; 200; TCP; 1234 → 1077
Inoltre, la tabella di s2 diventa

s2

Priority	Match	Action
0	*	Output(CONTROLLER,128)
1	eth_type = ETH_TYPE_MPLS mpls_label = 1000	DecMplsTtl Output(3)
3	eth_type = ETH_TYPE_IP ipv4_src=IP2 ipv4_dst=IP5 ip_proto = IPPROTO_TCP tcp_dst = 53	SetField(tcp_dst=1077) Output(3)
3	eth_type = ETH_TYPE_IP ipv4_src=IP5 ipv4_dst=IP2 ip_proto = IPPROTO_TCP tcp_src = 1077	SetField(tcp_src=53) Output(1)

(e) L'host destinatario risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow s1$: MAC5 → MAC2; IPv4; IP5 → IP2; 200; TCP; 53 → 1234

L'host h1, invia un datagramma UDP porta 7654 a h5 porta 8888.

(f) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

Soluzione:
$s1 \rightarrow s2$: MAC1 → MAC5; MPLS (1000); IP1 → IP5; 200; TCP; 7654 → 8888
Inoltre la tabella di s1 diventa

s1

Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = ETH_TYPE_IP ipv4_dst = IP5 ip_proto = IPPROTO_UDP udp_dst = 123	eth_dst = MAC3 ipv4_dst = IP3 DecNwTTL Output(3)
3	eth_type = ETH_TYPE_IP ipv4_dst = IP5 ip_proto = IPPROTO_UDP udp_dst = 8888	PushMpls SetField(mpls_label=1000) Output(3)

(g) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow s3$: MAC1 $\rightarrow$ MAC5; MPLS (1000); IP1 $\rightarrow$ IP5; 199; TCP; 7654 $\rightarrow$ 8888

(h) Scrivere il contenuto finale delle tabelle openflow.

Soluzione:
vedi sopra

Esercizio 2

Una matrice di commutazione crossbar di uno switch 4×4 usa un algoritmo per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza L,

Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto.

Ingresso 1: 4, 2, 1

Ingresso 2: 1, 2, 2

Ingresso 3: –, 4, 2

Ingresso 4: –, 4, 1

(a) Svolgere l'algoritmo Take-a-Ticket (TaT) fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi e quali ticket sono assegnati. L'algoritmo di assegnazione dei ticket privilegia le interfacce di ingresso secondo la seguente regola:

- le interfacce di uscita con identificativo dispari privilegiano le interfacce di ingresso con l'identificativo minore
- le interfacce di uscita con identificativo pari privilegiano le interfacce di ingresso con l'identificativo maggiore

Soluzione:

Round		Ingresso 1	2	3	4
1	Assegnazione	(1,1)	(2,2)	(2,1)	(1,2)
	Trasmissione	1	–	2	–
2	Assegnazione	(2,3)	–	(4,1)	–
	Trasmissione	–	2	4	1
3	Assegnazione	–	(2,4)		(4,2)
	Trasmissione	2	–		4
4	Assegnazione	(4,3)	–		
	Trasmissione	4	2		
5	Assegnazione		(1,3)		
	Trasmissione		1		

(b) Si consideri il caso in cui i pacchetti in entrata dalla coda 3 abbiano tutti lunghezza $2L$. Ripetere la simulazione dell'algoritmo TaT.

Soluzione:

Round		Ingresso 1	2	3	4
1	Assegnazione	(1,1)	(2,2)	(2,1)	(1,2)
	Trasmissione	1	–	2	–
2	Assegnazione	(2,3)	–	...	–
	Trasmissione	–	–	...	1
3	Assegnazione	–	–	(4,2)	(4,1)
	Trasmissione	–	2	–	4
4	Assegnazione	–	(2,4)	–	
	Trasmissione	2	–	4	
5	Assegnazione	(4,3)	–	...	–
	Trasmissione	–	2	...	
6	Assegnazione	–	(1,3)		
	Trasmissione	4	1		

Facendo riferimento all'algoritmo TaT con assegnazione casuale, si consideri uno scenario di traffico uniforme e code sempre sature.

(c) Qual è la probabilità che siano usate tutte le uscite?

Soluzione:

$$p = \frac{4!}{4^4} \simeq 9\%$$

(d) Qual è la probabilità che sia usata una sola uscita?

Soluzione:

$$p = \frac{4}{4^4} \simeq 1{,}5\%$$

(e) Qual è la probabilità che sia usata l'uscita numero 3?

Soluzione:

$$p = 1 - \left(1 - \frac{1}{4}\right)^4 \simeq 70\%$$

Esercizio 3

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Source Address	Destination Address	Source Port	Destination Port	Protocol
16.0.0.0/4	44.0.0.0/6	123	*	TCP
44.0.0.0/6	16.0.0.0/5	*	22	TCP
24.0.0.0/5	44.0.0.0/6	*	53	UDP
*	24.0.0.0/5	*	123	UDP
16.0.0.0/5	*	123	*	UDP
16.0.0.0/4	16.0.0.0/4	*	80	*

(a) Definire il bitvector set per ciascun campo.

Soluzione:

Source Address	
16.0.0.0/4	100101
16.0.0.0/5	100111
24.0.0.0/5	101101
44.0.0.0/6	010100
*	000100

Destination Address	
16.0.0.0/4	000011
16.0.0.0/5	010011
24.0.0.0/5	000111
44.0.0.0/6	101010
*	000010

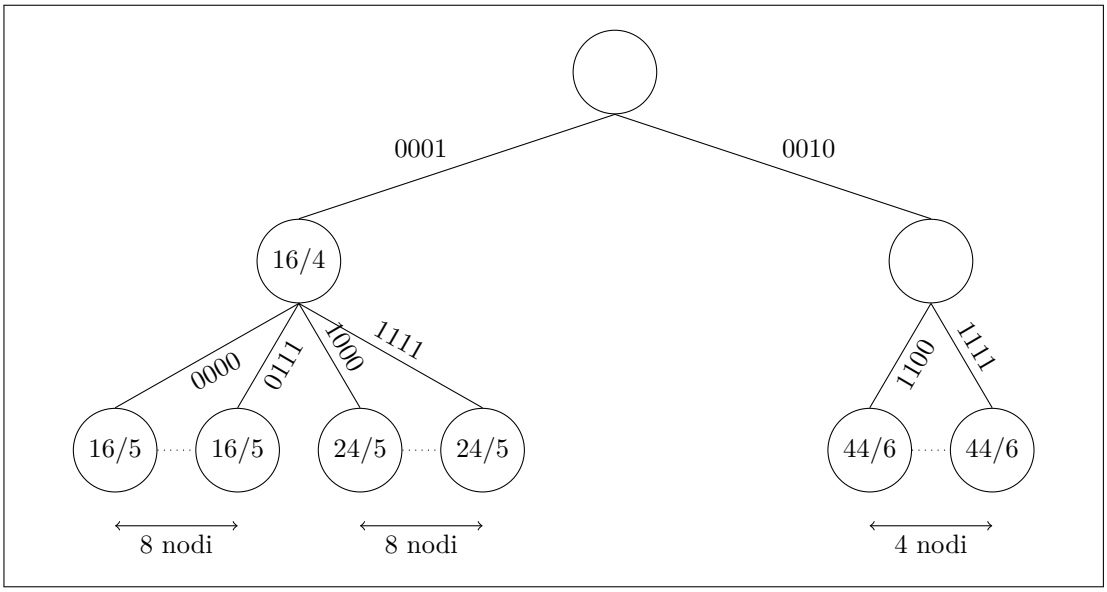
Source Port	
123	111111
*	011101

Dest. Port	
22	110010
53	101010
123	100110
80	100011
*	100010

protocol	
TCP	110001
UDP	001111
*	000001

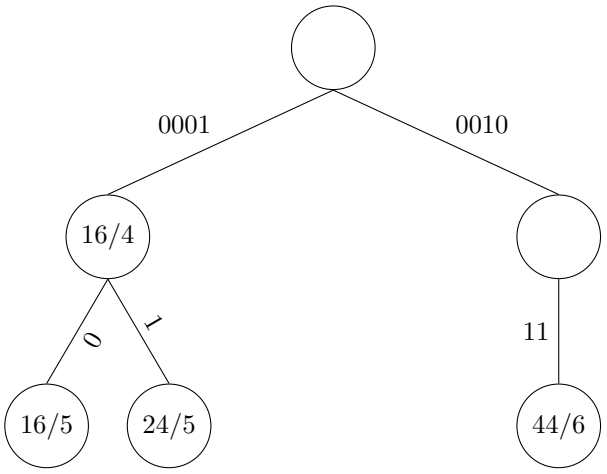
(b) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando un multibit trie a passo 4. Disegnare il trie.

Soluzione:



(c) L'algoritmo di ricerca del prefisso IP sorgente (prima colonna) è implementato usando un multibit trie a passo variabile. Scegliere i passi in modo da avere un numero di nodi inferiore a quelli ottenuti nel caso precedente. Disegnare il trie.

Soluzione:



(d) Si considerino i pacchetti (64.10.0.1, 96.0.0.2, 80, 53, UDP) e (16.50.0.1, 16.0.0.2, 80, 80, TCP). Applicare ad entrambi la procedura di BVLS per identificare gli eventuali match.

Soluzione:
Prer il primo pacchetto nessuna corrispondenza. Per il secondo pacchetto si applica le regola 6.

Esercizio 4

Un sistema di controllo degli accessi conserva una black list di indirizzi MAC cui è vietato l'accesso in rete. La black list usa un filtro di Bloom costituito da un array di 1024 bit e 4 funzioni hash.

(a) Calcolare la probabilità di falso positivo dopo aver effettuato l'inserimento di 300 indirizzi MAC

Soluzione:

$$(1 - \exp(-4 \times 300/1024))^4 = 23\%$$

(b) Calcolare il numero massimo di inserimenti per avere un probabilità di falso positivo di 10^{-4}

Soluzione:

$$(1 - \exp(-4n/1024))^4 < 10^{-4}$$

da cui $n < -\frac{1024 \log 0.9}{4}$, ovvero $n \leq 26$

(c) Siano le funzioni di hash definite come

$$h_i(x) = \lfloor 2^{-8i} x \rfloor \bmod 1024 \quad \text{per } 1 \leq i \leq 4$$

Dire se tali funzioni hash sono una buona scelta oppure presentano problemi anche in relazione alla natura del dato inserito.

Soluzione:
Le funzioni non sono indipendenti. Inoltre gli input sono indirizzi MAC, che non sono scelti uniformememente a caso.

Si considerino le funzioni hash

$$h_i(x) = (2i + 1)x \bmod 1024 \quad \text{per } 1 \leq i \leq 4$$

(d) Inizialmente tutti i bit dell'array sono posti a zero. Inserire gli elementi 33 e 99 nel filtro di Bloom. Mostrare lo stato di ogni bit prima e dopo ogni inserimento.

Soluzione:
Primo inserimento, setta i bit: 99, 165, 231, 297.
Secondo inserimento setta i bit: 297, 465, 693, 891.
Stato finale: sono settati i bit 99, 165, 231, 297, 465, 693, 891.

(e) Conclusi gli inserimenti, verificare l'appartenza al dataset dell'elemento 231.

Soluzione:
$h_2(x) = 131$ che non è settato. Quindi l'elemento non è presente.